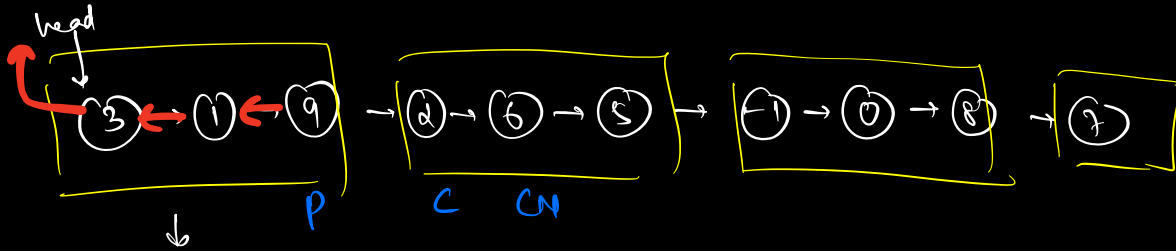


Reverse in buckets of k



o/p: 9 → 1 → 3 → 5 → 6 → 2 → 8 → 0 → 4 → 7

↑
head

Node reverseK(head, k) {

 int c = k

 curr = head

 prev = null

 CN = null

 while (curr != null & c > 0) {

 CN = curr.next

 curr.next = p

 p = curr

 curr = CN

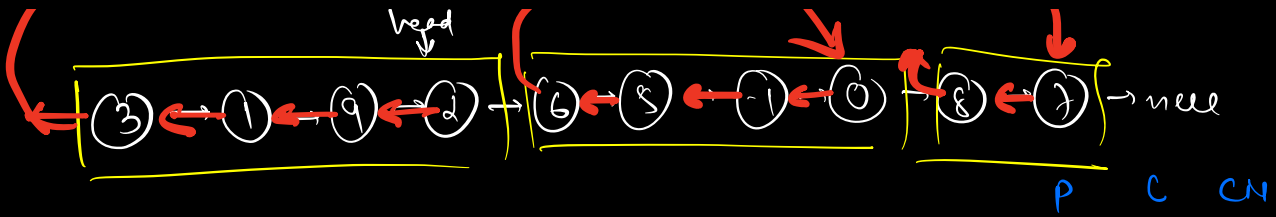
 c--

 }

 head.next = reverseK(curr, k)

 return p

}



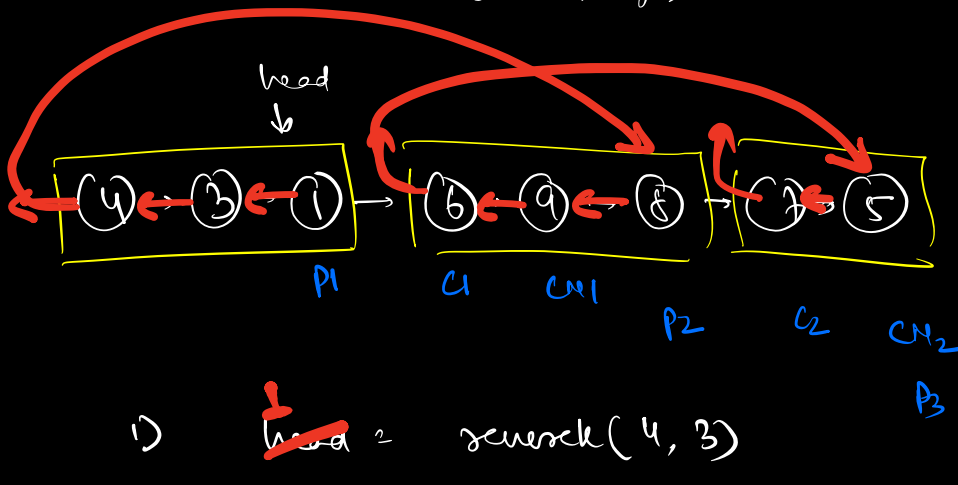
reverse(3, 4)

reverse done

3.next = reverse(6, 4)

6.next = reverse(8, 4)

6.next = 7



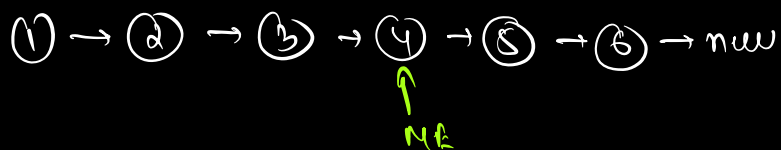
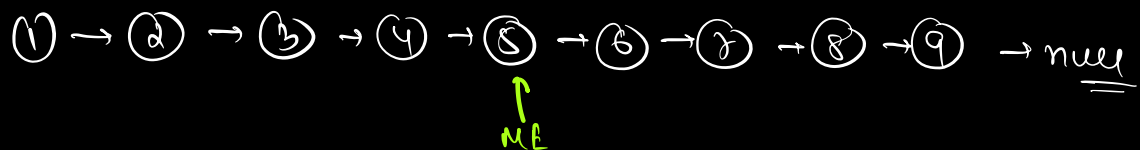
1) ~~head~~ = reverse(4, 3)

4.next = reverse(6, 3)

6.next = reverse(7, 3)

TC $\Rightarrow O(N)$
SC $\Rightarrow O(N/10)$

Q1. Find the middle element of a LL $\begin{cases} \text{odd} \rightarrow \frac{N+1}{2} \\ \text{even} \rightarrow \frac{N}{2} + 1 \end{cases}$



Basic

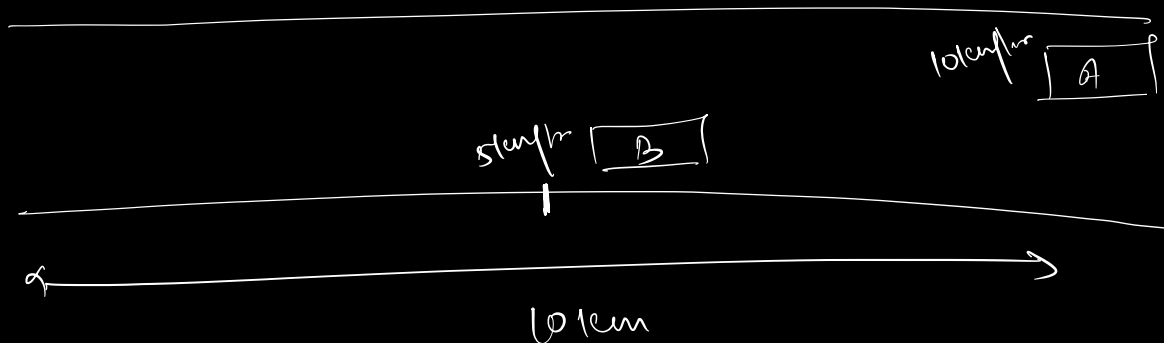
i) find the length of the LL → l

ii) iterate to $\left(\frac{N}{2} + 1\right)$ → return node

TC → O(N) ← 2 iterations

SC → O(1)

Optimised



A $\boxed{2x}$

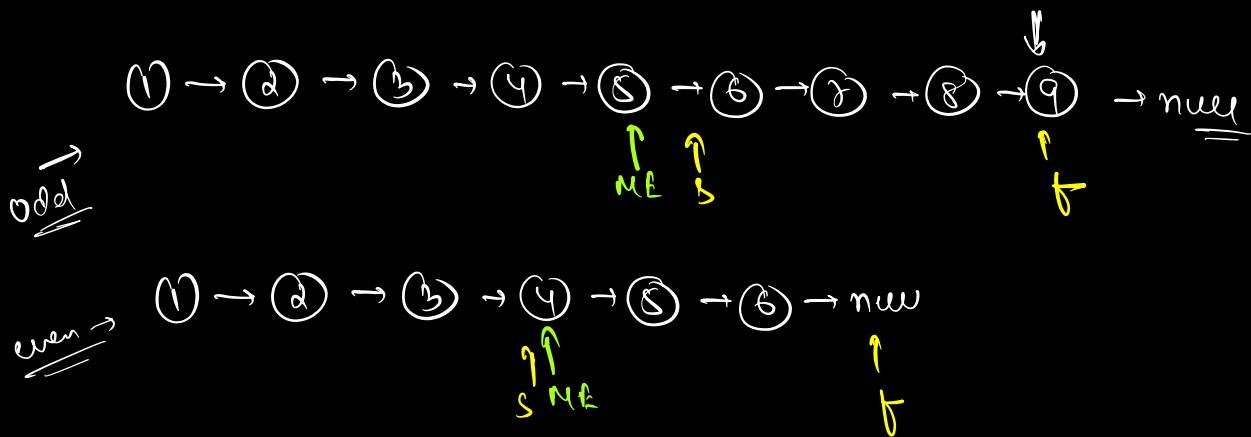
B $\boxed{x}$



Time taken car A to cover the road is $\frac{l}{2x}$

In same time, how much dist will car B cover

$$\Rightarrow x \times \frac{l}{2x} \Rightarrow \left(\frac{l}{2}\right)$$



slow = head

fast = head

while (fast != null & fast.next != null) {

slow = slow.next

fast = fast.next.next

}

return slow

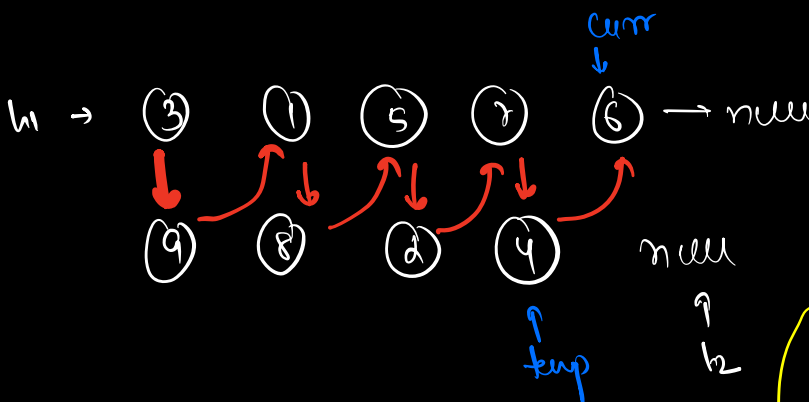
$TC = O(N)$ ← 1 iteration
 $SC = O(1)$

Q2. Merge 2 LL $(1 \rightarrow 2 \rightarrow 1 \rightarrow 2 \rightarrow 1)$

$h_1 \rightarrow (3) \rightarrow (1) \rightarrow (5) \rightarrow (7) \rightarrow (6) \rightarrow \text{null}$

$h_2 \rightarrow (9) \rightarrow (8) \rightarrow (2) \rightarrow (4) \rightarrow \text{null}$

O/p $\rightarrow (3) \rightarrow (9) \rightarrow (1) \rightarrow (8) \rightarrow (5) \rightarrow (2) \rightarrow (7) \rightarrow (4) \rightarrow (6) \rightarrow \text{null}$



$TC = O(N)$
 $SC = O(1)$

H.W. solve when

no such

relationship

$\& \text{size}(h_1) > \text{size}(h_2)$

while($h_2 \neq \text{null}$) {

temp = h_2

$h_2 = h_2 \rightarrow \text{next}$

temp.next = curr.next

curr.next = temp

curr = temp.next

}

U

① → ② → ③ → ④ → ⑤ → ⑥ → ⑦ → ⑧ → ⑨ → null

O/p $\Rightarrow$ $(1) \rightarrow (9) \rightarrow (2) \rightarrow (8) \rightarrow (3) \rightarrow (7) \rightarrow (4) \rightarrow (6) \rightarrow (5) \rightarrow \text{null}$

⑤ → ① → ② → ③ → ④ → ⑤ → null

$(n_2) - (9) \rightarrow (8) \rightarrow (7) \rightarrow (6) \rightarrow \text{null}$

```

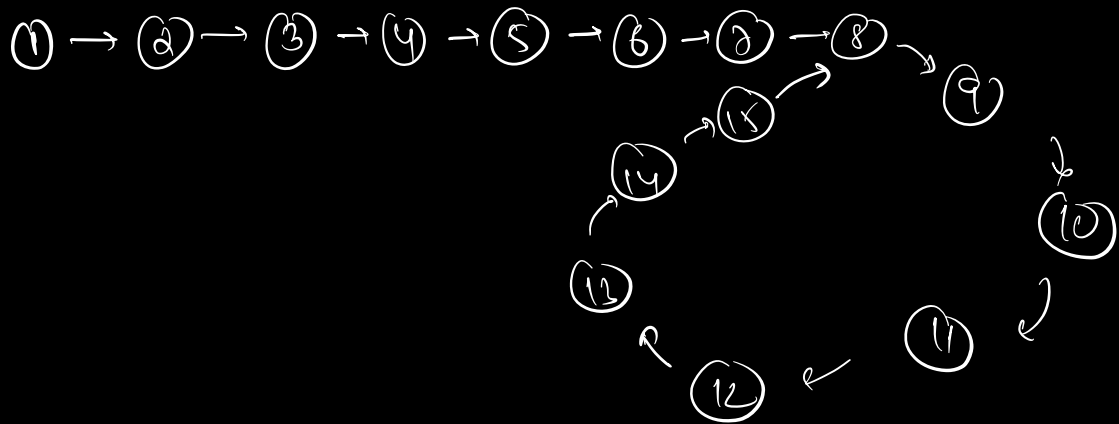
Node m = middle(head);
Node h2 = m.next;
m.next = null;

h2 = reverse(h2);

h = merge(h, h2);

```

Q 4. LL detect if there is a cycle in LL



slow, fast $\Rightarrow$ if these ptrs ever meet, then there definitely is a loop.

slow = head

fast = head

while (fast != null && fast->next != null)

slow = slow->next

fast = fast->next->next

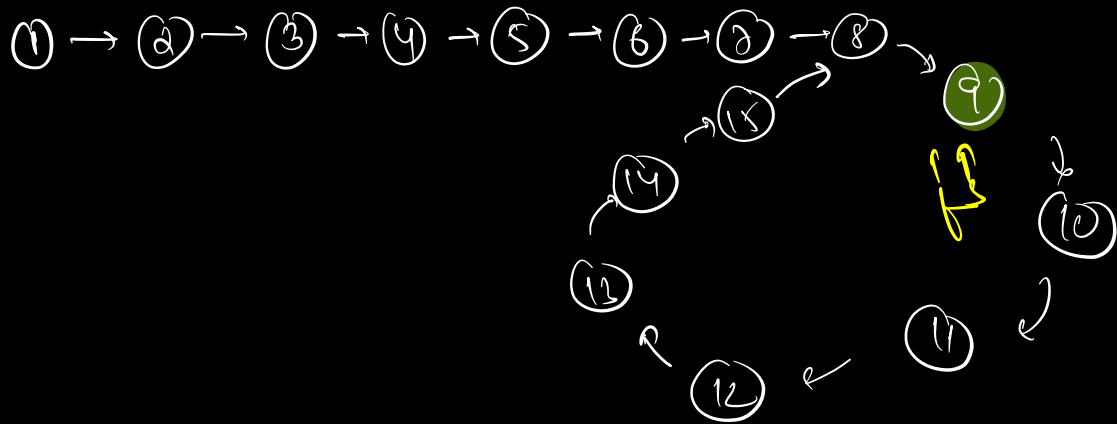
if (slow == fast)

return true

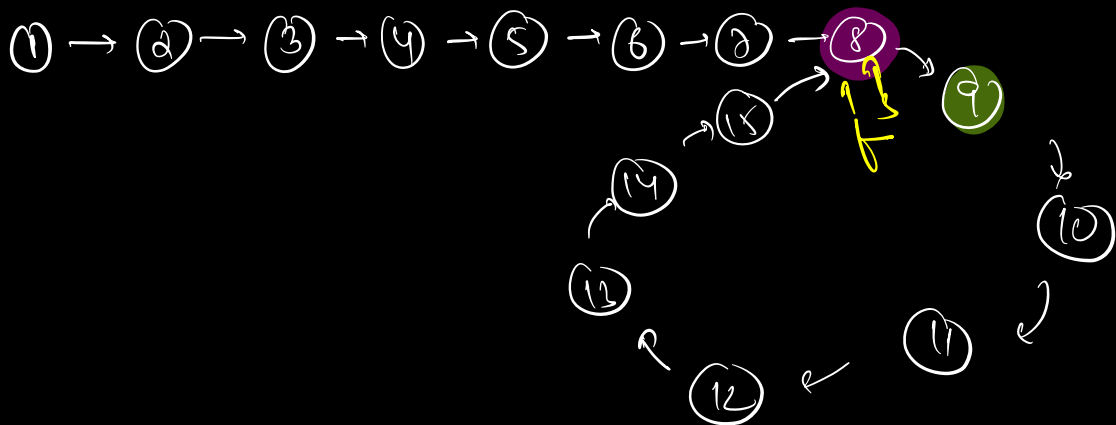
}

return false

}



* find the starting pt. of the loop:-



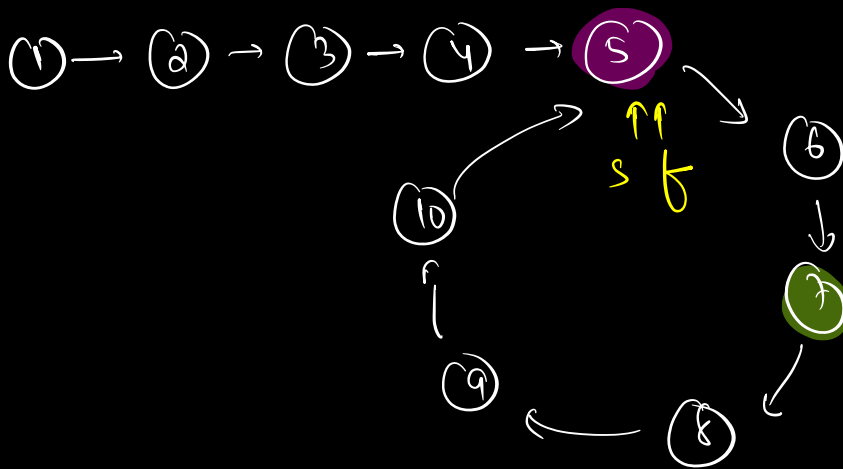
i) slow, fast → to detect the loop

ii) if they meet, loop exists

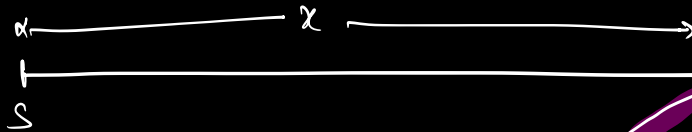
iii) move slow ptr to head

iv) move both ptr with 1 speed,

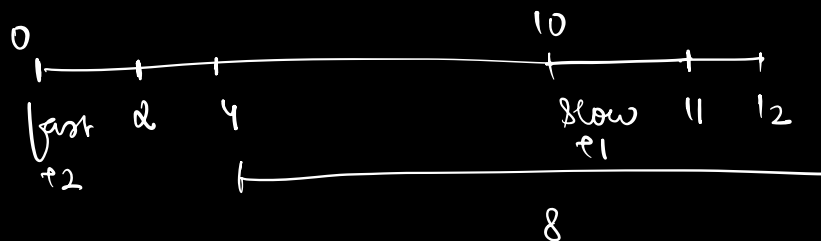
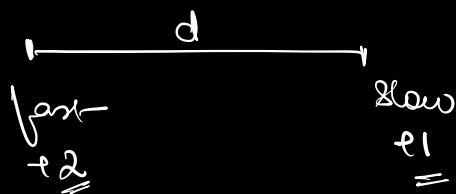
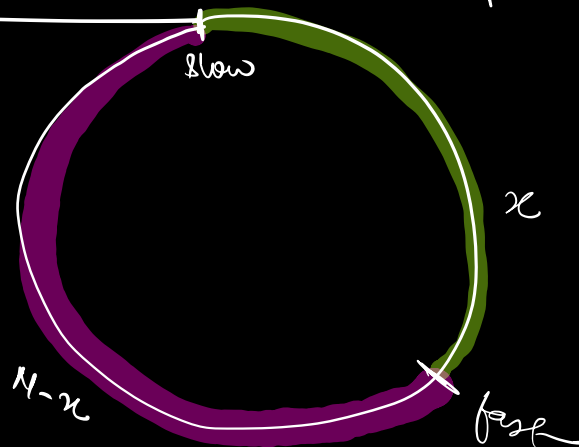
→ wherever they will meet → start of cycle



$(x < N)$



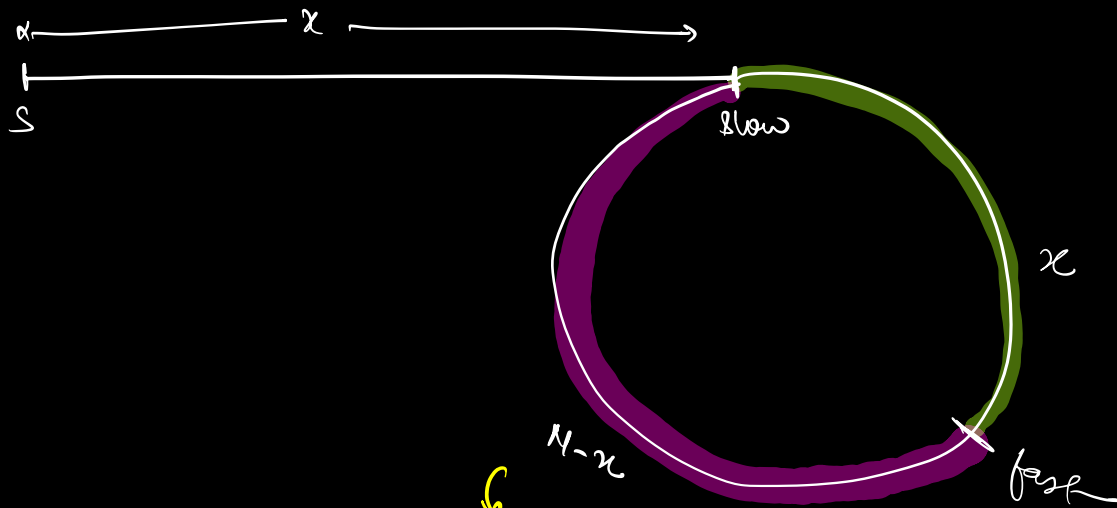
length of loop = N



at each step/iteration $\rightarrow$ slow & fast ptr come closer to each other by 1 unit

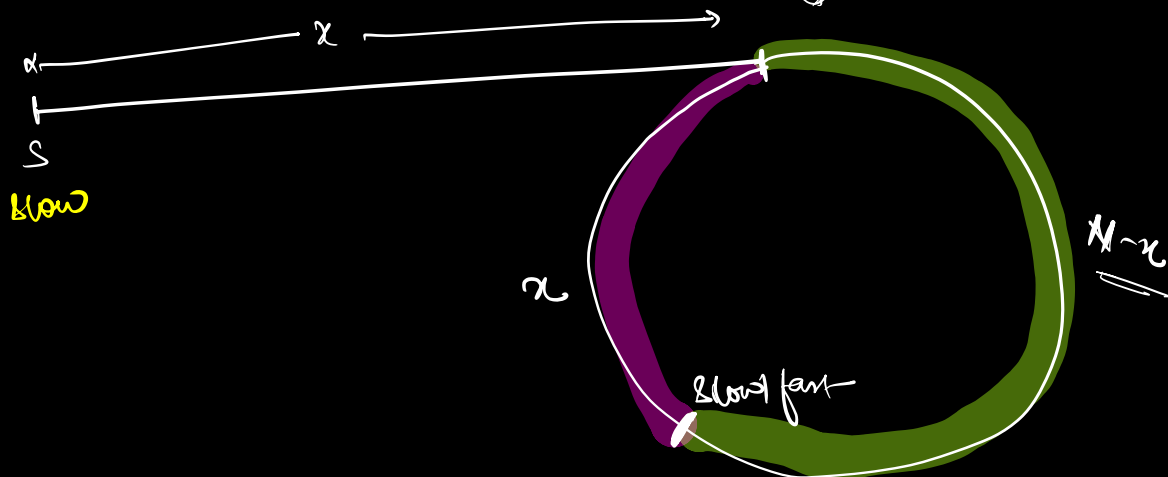
→ if initial b/w slow & fast → d

they use d steps/iterations to meet

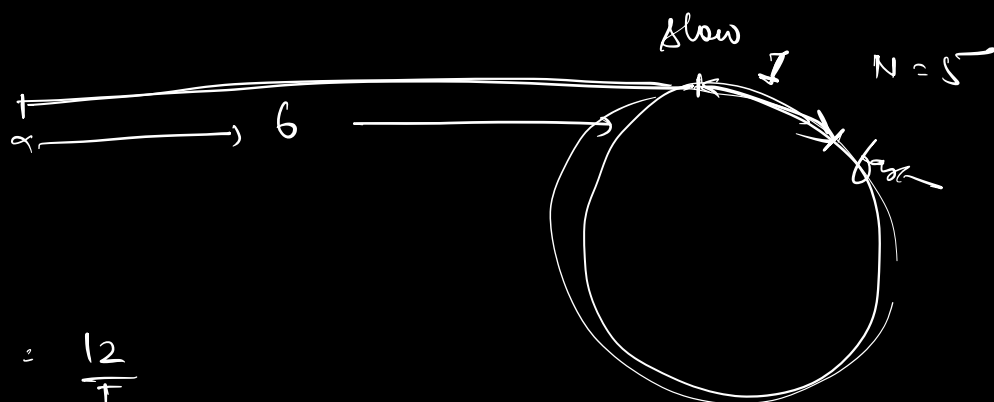
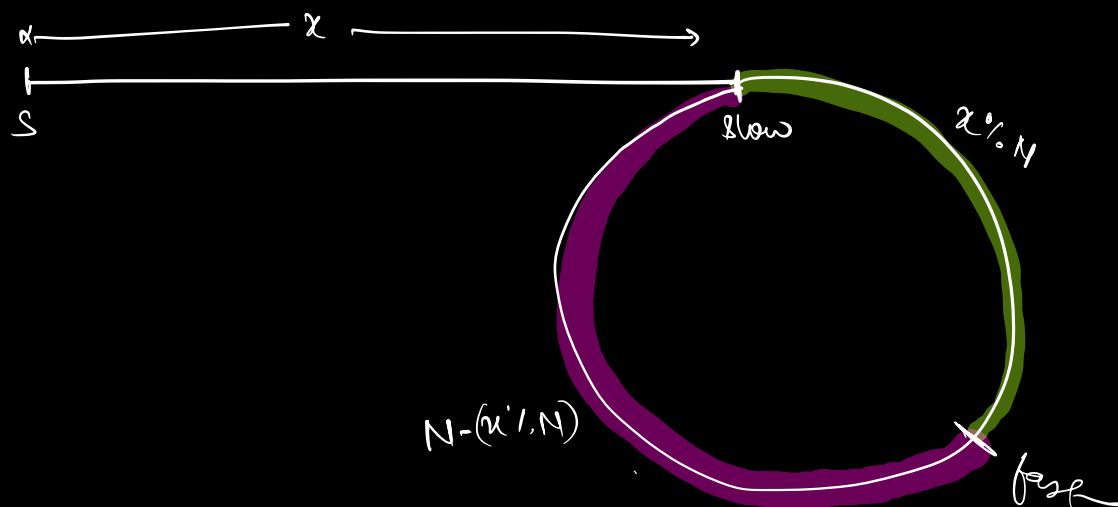


it will need
(N-x) steps for
fast & slow
to meet

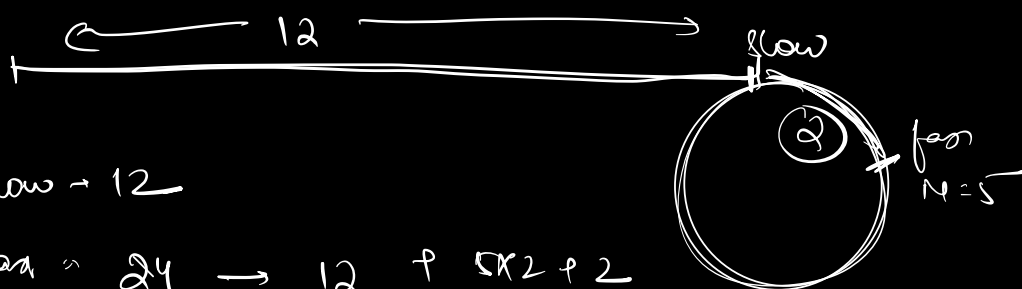
after (N-x) steps



$x > N$



fast = $\frac{12}{5}$
6 +
(non-loop)



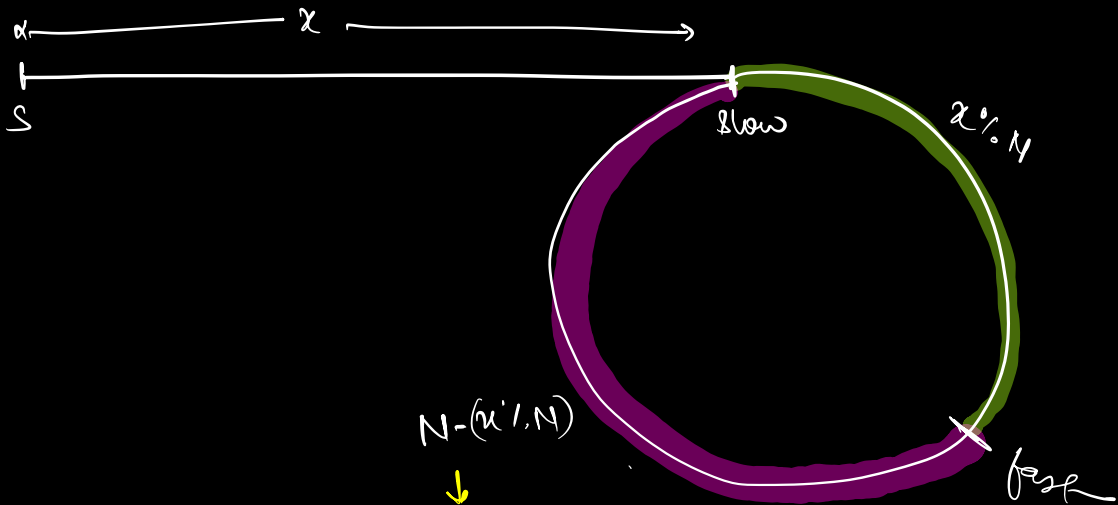
slow = 12

fast = 24 → 12 + 5 * 2 + 2
(non-loop)

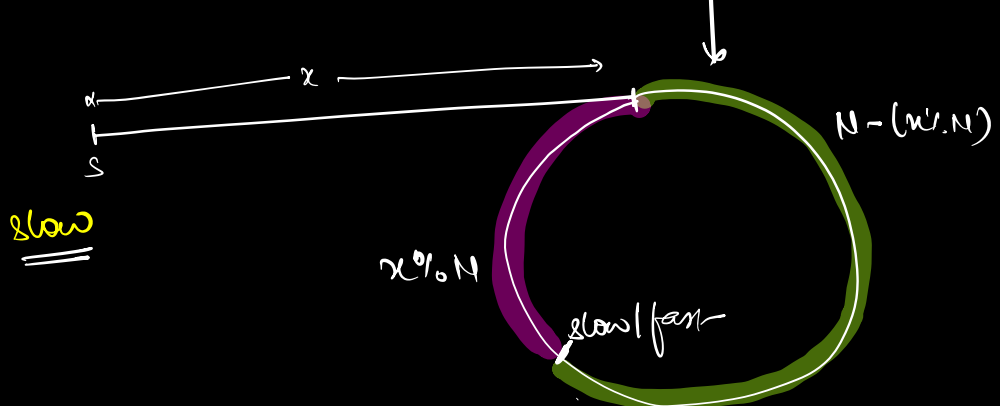
$$[x = N \times q + x' \wedge N]$$

$\downarrow$
 rotations
of loop

$\downarrow$
 remaining
dist



$\downarrow$
 they will need
 $[N - (x', N)]$ steps to
 meet



$$\underline{x} = \overset{\text{no change}}{\cancel{N \times q}} + \underline{x \% N}$$

$$x - N \times q = x \% N$$

H.W

- i) detect a loop in LL
- ii) find starting pt. of cycle in LL
- iii) remove the cycle from LL